The main objective of the workshop was to build a neural network classifier that can determine if an image is a muffin or a chihuahua. This was completed in the steps of first building the neural network, loading the data, training the model on the data, and then visualizing the results.

In this workshop we built a convolutional neural network (CNN) when defining the MySkynet class. This is the most important step of the workshop as the CNN works like a human eye, looking for small patterns before identifying the whole object (Xu, 2025). Image classification was a main concept used as it was the task of the model labeling the image as chihuahua or muffin. We used the datasets.ImageFolder tool to organize the photos to help the model learn them to carry out image classification. PyTorch defines tensors as a specialized data structure that are very similar to arrays and matrices (PyTorch, 2025). In the load or data section of this workshop we used transforms.ToTensor() to turn the images into these mathematical containers (PyTorch, 2025). ReLU (Rectified Linear Unit) applies the rectified linear unit function element-wise (PyTorch, 2025). In this workshop we used this function in the build your neural network. We also touched on transfer learning which allowed these advanced principles to achieve high accuracy without needing millions of original photos (Xu, 2025).

The biggest challenge I faced was finding the correct tweaks to achieve close to 100% accuracy. When first adjusting the model with a learning rate of 0.001, the accuracy remained stuck around 70%. To overcome this, I adjusted the learning rate to 0.01. This allowed the model to learn more quickly. This change, combined with increasing the layer dimensions, eventually pushed the model to a final validation accuracy of 93.33%.

Working through this lab provided a clear insight into the relationship between model capacity and performance. I learned that simply having a neural network is not enough. The architecture must be wide enough to capture complex features. By increasing the hidden units in the linear layers to 256, 128, and 64, I gave the model more room to store the patterns it detected. I also gained a better understanding of how computers interpret visual data. While I see a dog or a muffin, the computer only sees numbers. The success of the classification depends entirely on how well the mathematical transforms and activation functions like ReLU filter those numbers into meaningful predictions.

The techniques learned in this workshop have much more potential beyond simple image labeling. In the medical field CNNs are used to distinguish between healthy tissue and malignant tumors in X-rays or MRIs. These techniques are also being used in autonomous vehicles, to classify objects like pedestrians, stop signs, and other cars in real time to make safe driving decisions. As noted by Xu (2025), these advantages in feature

extraction make CNNs indispensable in fields like facial recognition and medical impact analysis.

This lab was a very fun and insightful look into how to train computers to distinguish and categorize images of different items. The technical terms were overwhelming at first but seeing them in action, and doing the research to complete this journal has made the concepts much clearer. It was fun to see something that is so simple for me to distinguish from the view point of a computer that can only map out edges and shapes. Going through the computers entire learning process helped me realize how this seemly simple task becomes a high stakes classification for a computer. It made me think deeply about how complex the AI tools being used daily really are. It was very rewarding making the adjustments and being able to achieve 93.33% accuracy!

Sources:

PyTorch. (2025). ReLU. PyTorch Documentation. https://docs.pytorch.org/docs/stable/generated/torch.nn.ReLU.html

PyTorch. (2025). Tensors. PyTorch Documentation. https://docs.pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html

PyTorch. (2025). Transforms. PyTorch Documentation. https://docs.pytorch.org/tutorials/beginner/basics/transforms_tutorial.html

PyTorch. (2025). SGD. PyTorch Documentation. https://docs.pytorch.org/docs/stable/generated/torch.optim.SGD.html

Xu, J. (2025). Understanding Convolutional Neural Networks for Image Classification. Transactions on Computer Science and Intelligent Systems Research. https://www.researchgate.net/publication/394870797